

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS

SUPERIORES DE MONTERREY

Operaciones de aprendizaje automático (TC5061)

Código de experimentación base con MLflow

Carlos Noel Ojeda Angulo, A01741085

11 de Octubre de 2026

1. Entrega y repositorio

Repositorio individual (accesible para el profesor): **https://github.com/Carlos-Ojeda/TC5061.10** Contiene train.py, requirements.txt, el dataset, el notebook de exploración y este README de ejecución.

El notebook de la semana anterior se conserva solo para el análisis exploratorio. La carga, la limpieza, el entrenamiento, la evaluación y el registro viven ahora en un script ejecutable, organizado en funciones con una responsabilidad cada una: parse_args, load_dataset, clean_dataset, split_features, build_pipeline, evaluate_regression y log_run.

2. Comando de ejecución

El script corre con valores por defecto si no se pasa ningún argumento. Los hiperparámetros expuestos son --alpha, --fit-intercept / --no-fit-intercept, --solver y la semilla --seed.

```
python train.py
```

Corridas comparadas bajo el experimento TC5061_Wine_Quality_Experimentacion:

```
python train.py --run-name ridge_default
python train.py --alpha 0.01 --run-name ridge_alpha_0.01
python train.py --alpha 10 --run-name ridge_alpha_10
python train.py --alpha 100 --run-name ridge_alpha_100
python train.py --no-fit-intercept --run-name ridge_sin_intercepto
python train.py --alpha 1 --solver saga --run-name ridge_solver_saga
```

Interfaz de comparación:

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
```

3. Instrumentación y reproducibilidad

Cada corrida abre mlflow.start_run() y registra los hiperparámetros con log_param, tres métricas de prueba (MAE, RMSE y R^2) con log_metric, dos artefactos (gráfica de residuos y metrics.json) con log_artifact, y el pipeline entrenado con mlflow.sklearn.log_model. El tracking usa SQLite para evitar el FileStore bloqueado en MLflow 3.16. La semilla 42 se fija en random, NumPy, train_test_split y Ridge. Dos ejecuciones con los mismos argumentos (ridge_default y ridge_default_repro) produjeron exactamente MAE = 0.506626, RMSE = 0.652344 y R^2 = 0.365636. Las versiones quedan ancladas en requirements.txt (pandas 3.0.6, scikit-learn 1.9.1, MLflow 3.16.1, matplotlib 3.11.2).

4. Análisis de resultados

Se partió del baseline de regresión lineal del notebook (vino tinto de UCI, 1 359 filas tras eliminar 240 duplicados, partición 80/20 estratificada). LinearRegression no expone hiperparámetros suficientes para una comparación sistemática, así que el script usa Ridge: es el sucesor natural del OLS y permite variar la regularización L2 sin cambiar el resto del pipeline (imputación por mediana y StandardScaler).

La mejor configuración fue ridge_alpha_0.01: alpha = 0.01, intercepto activo, solver auto y seed = 42. Obtuvo MAE = 0.5066, RMSE = 0.6523 y R^2 = 0.3657. El default (alpha = 1.0) quedó a milésimas (MAE = 0.5066, R^2 = 0.3656) y saga reprodujo el mismo desempeño. Subir alpha a 10 y 100 empeoró de forma monótona el error (MAE 0.5073 y 0.5135). Quitar el intercepto derrumbó el modelo: MAE = 5.6126 y R^2 = −46.59. Por tanto, el hiperparámetro más influyente no fue alpha, sino fit_intercept. En un objetivo como quality, centrado cerca de 5–6 y no en cero, el sesgo del modelo es indispensable; la regularización L2 apenas mueve las métricas en este rango porque las 11 variables ya están estandarizadas y el problema es poco sensible a un ridge suave.

Limitaciones. Primero, quality es ordinal (enteros de 3 a 8) y se modeló como continuo: el residual se agrupa en bandas y el R^2 ronda 0.37, igual que el OLS del notebook. Segundo, se usó una sola partición y no validación cruzada, así que la ventaja de alpha = 0.01 sobre alpha = 1.0 puede ser ruido de esa muestra. Tercero, el experimento cubre solo vino tinto y un modelo lineal; no explora interacciones ni no linealidades. La evidencia registrada, sin embargo, ya basta para descartar con seguridad la variante sin intercepto y para no perseguir alphas grandes: el siguiente paso debería ser otro tipo de modelo (por ejemplo un ensamble), no más búsqueda dentro de Ridge.

5. Evidencias de MLflow

Las capturas siguientes documentan la tabla comparativa del experimento y el detalle de la mejor corrida, incluyendo parámetros, métricas y el artefacto de residuos.

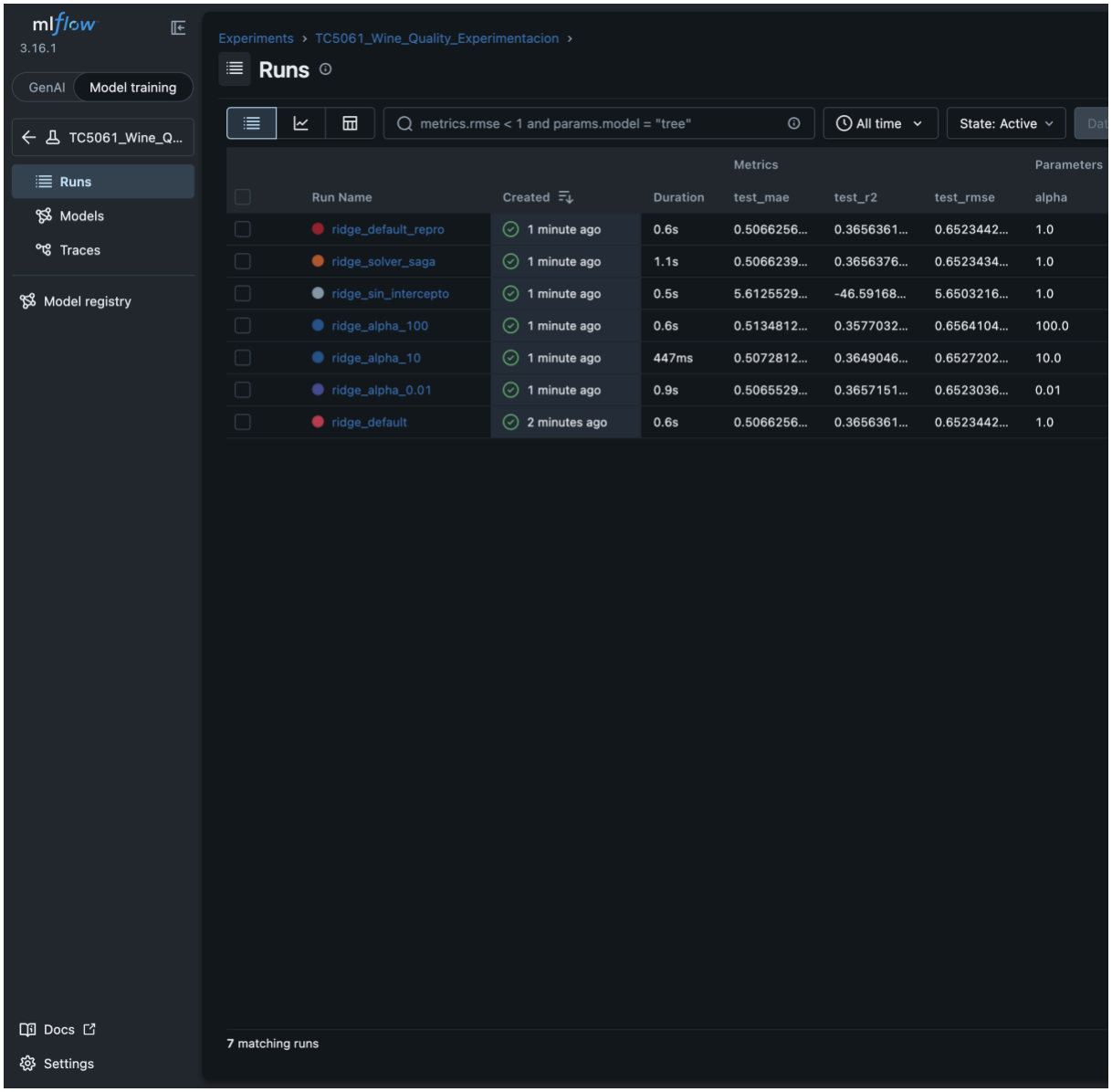


Figura 1. Vista de comparación de las siete corridas en MLflow Tracking. Se observan test_mae, test_r2, test_rmse y alpha bajo el experimento TC5061_Wine_Quality_Experimentacion.

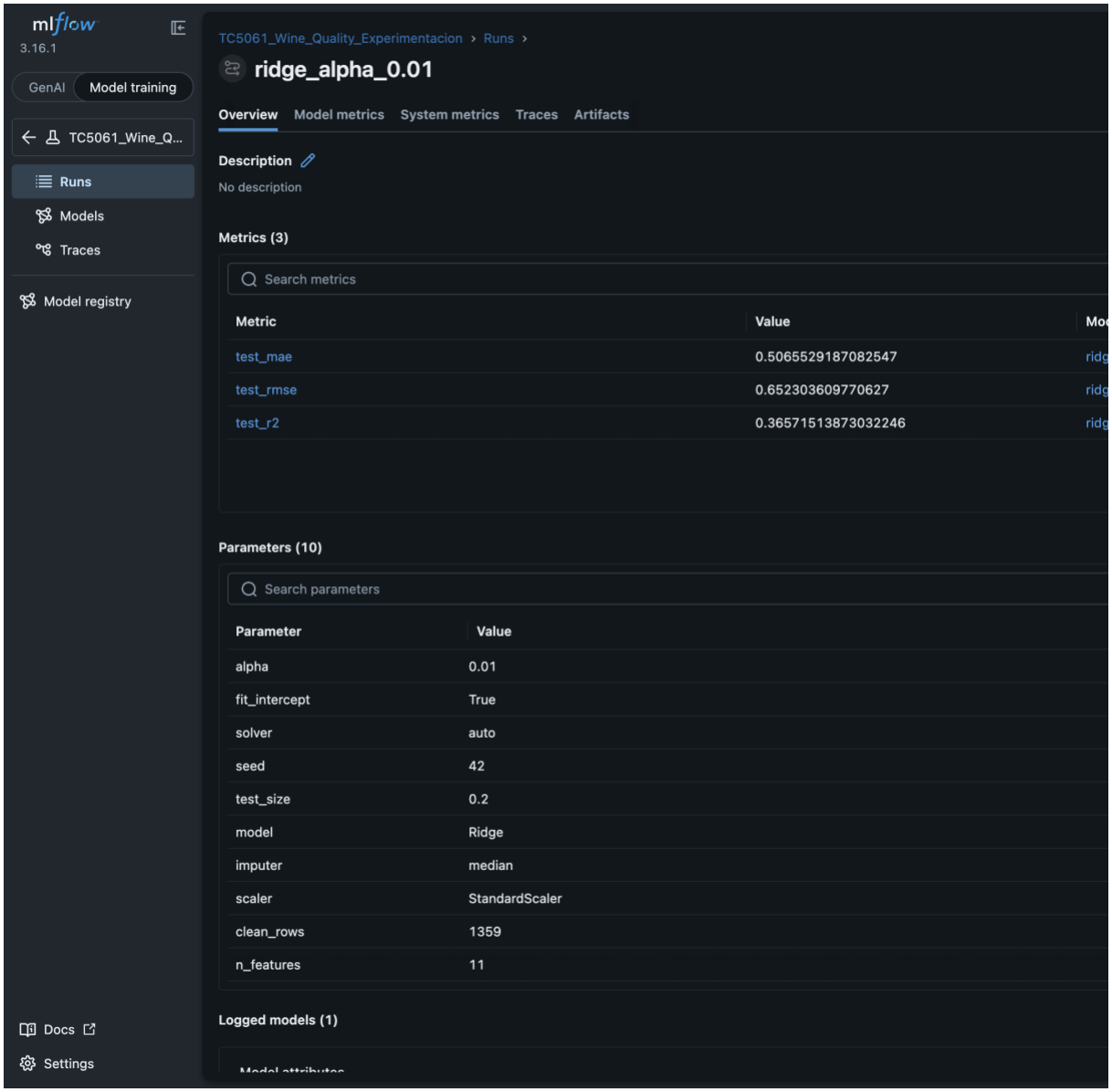


Figura 2. Detalle de la mejor corrida (ridge_alpha_0.01): tres métricas, diez parámetros y el modelo ridge_pipeline registrado.

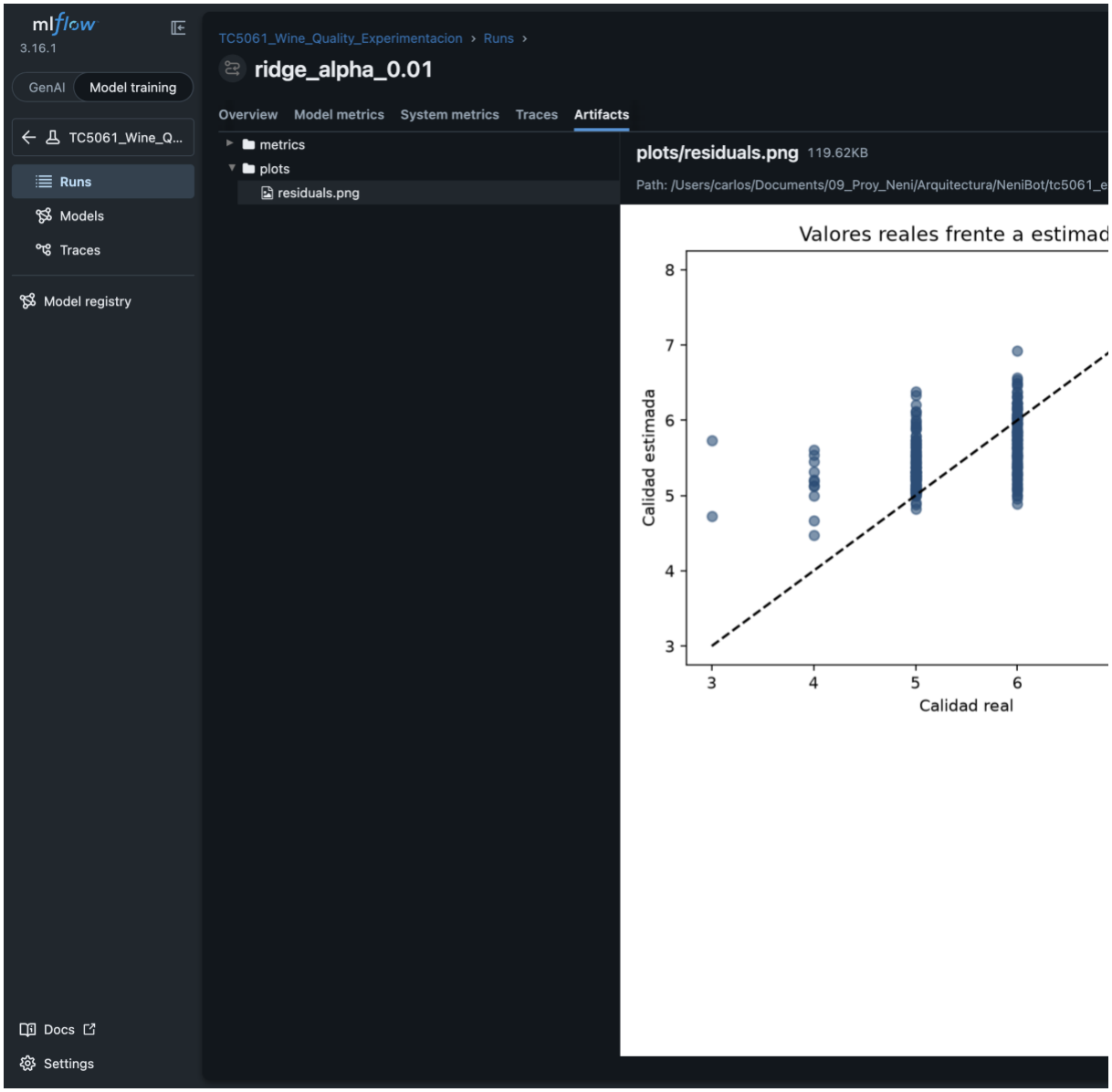


Figura 3. Pestaña Artifacts de la misma corrida: carpeta plots con residuals.png y vista previa de la gráfica registrada con log_artifact.